CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

Anesthesia is a fundamental component of modern surgical procedures, enabling complex medical operations by ensuring that patients remain unconscious, pain-free, and physiologically stable throughout the surgical process. The administration of anesthesia involves a delicate balance between maintaining adequate depth of anesthesia and ensuring patient safety. Even slight deviations in physiological parameters such as oxygen saturation (SpO_2), heart rate, temperature, or anesthetic gas concentration can lead to severe complications, including hypoxia, cardiovascular instability, or intraoperative awareness.

In conventional operating room environments, anesthesia monitoring is performed using specialized workstations equipped with multiple sensors and display systems. These systems provide real-time information about the patient's vital signs and anesthetic gas levels. However, despite technological advancements, most existing systems rely heavily on continuous manual supervision by anesthesiologists.[1] This dependence increases workload, introduces the possibility of human error, and may lead to delayed response during critical situations.

With the rapid advancement of embedded systems, sensor technologies, and Internet of Things (IoT) platforms, there is a growing opportunity to develop intelligent healthcare monitoring systems that enhance efficiency and reliability.[2] IoT-enabled systems allow real-time data acquisition, remote monitoring, cloud storage, and automated alert mechanisms. These capabilities significantly improve the ability to detect abnormalities and respond quickly to potential risks.

In this context, the proposed project titled *"ESP32-Based IoT Anesthesia Monitoring and Assistive Gas Flow Control System"* aims to integrate multiple physiological and environmental sensors with an embedded microcontroller to create a smart monitoring

platform. The system continuously measures parameters such as SpO_2, heart rate, temperature, gas concentration, pressure, and flow rate. It also incorporates alert mechanisms and assistive gas flow control using a solenoid valve.

The integration of IoT enables remote monitoring and data logging, allowing healthcare professionals to observe patient conditions from distant locations. Additionally, the system is designed to be cost-effective and modular, making it suitable for training environments, research applications, and resource-constrained healthcare settings.

1.2 NEED FOR THE STUDY

The need for this study arises from the limitations of traditional anesthesia monitoring systems and the increasing demand for intelligent healthcare solutions. Existing systems, although advanced, present several challenges:

- **High dependency on manual monitoring:** Continuous observation by anesthesiologists is required, increasing workload and fatigue
- **Delayed response to critical conditions:** Manual intervention may not be immediate in emergency situations
- **Limited remote accessibility:** Most systems operate locally without real-time remote monitoring capabilities
- **High cost and complexity:** Advanced anesthesia machines are expensive and not easily accessible in smaller hospitals

In addition, traditional systems often lack integrated data communication and intelligent decision-support mechanisms. While some systems provide alarms, they are typically limited to basic threshold detection and do not support remote notifications or long-term data analysis.

There is a growing need for a system that can:

- Automate monitoring and reduce human dependency

- Provide real-time alerts and faster response
- Enable remote monitoring through IoT
- Offer a cost-effective alternative for healthcare institutions

This study addresses these requirements by proposing a compact, embedded system that combines sensing, processing, communication, and control functionalities in a single platform.

1.3 MOTIVATION

The motivation behind this project stems from the critical importance of patient safety during anesthesia and the limitations of existing monitoring approaches. In surgical environments, even minor delays in detecting abnormal conditions such as gas leakage, pressure fluctuations, or irregular vital signs can result in serious consequences.

Several factors contributed to the development of this system:

- The need to reduce human workload in continuous monitoring
- The possibility of human error in manual supervision
- The increasing demand for smart and connected healthcare systems
- The availability of low-cost embedded platforms like ESP32
- The potential of IoT to enable remote supervision and telemedicine

Furthermore, many healthcare facilities, especially in developing regions, lack access to high-end anesthesia monitoring systems due to cost constraints. This creates a gap between required medical standards and available resources.

1.4 SCOPE OF THE WORK

The scope of this project focuses on the design and implementation of an embedded system for real-time anesthesia monitoring and assistive gas flow control. The system

integrates multiple sensors and components to monitor both patient vitals and environmental conditions.

The key functional scope includes:

- Measurement of physiological parameters such as:
    - Blood oxygen saturation (SpO_2)
    - Heart rate
    - Body temperature
- Monitoring of environmental and system parameters such as:
    - Gas concentration
    - Pressure levels
    - Flow rate of anesthetic gases
- Real-time display of data using an OLED module
- Implementation of alert mechanisms (buzzer and LEDs)
- Assistive control of gas flow using a solenoid valve
- IoT-based data transmission for remote monitoring

1.5 OBJECTIVES OF THE PROPOSED WORK

The primary objectives of the proposed system are as follows:

1. To design and develop an ESP32-based embedded system for anesthesia monitoring
2. To acquire real-time physiological data such as SpO_2 and heart rate
3. To monitor environmental parameters including gas concentration and pressure
4. To implement a real-time alert system for abnormal conditions
5. To develop an assistive mechanism for controlling anesthetic gas flow

6. To integrate IoT for remote monitoring and data transmission
7. To design a cost-effective and modular system suitable for practical applications

Secondary objectives include:

- Improving system responsiveness during emergency conditions
- Enhancing user interaction through visual and audio feedback
- Providing a scalable platform for future upgrades

1.6 ORGANIZATION OF THE THESIS

The thesis is structured to provide a systematic understanding of the proposed system and its implementation:

- **Chapter 1: Introduction**

Presents the background, need, motivation, objectives, and scope of the project

- **Chapter 2: Literature Review**

Discusses existing anesthesia monitoring systems and related technologies

- **Chapter 3: Methodology / System Design**

Describes the system architecture, IoT integration, and working flow

- **Chapter 4: Hardware Components**

Explains the components used and their roles in the system

- **Chapter 5: Software Components**

Details the programming structure, libraries, and implementation

- **Chapter 6: Implementation and Results**

Presents system operation, outputs, and observed performance

- **Chapter 7: Discussion and Conclusion**

Analyzes results and summarizes key findings

- **Chapter 8: Future Scope**

Suggests improvements and future enhancements .

CHPATER 2

LITERATURE REVIEW

2.1 INTRODUCTION

Anesthesia monitoring systems are critical medical technologies used during surgical procedures to ensure patient safety while anesthetic agents are administered. The primary objective of anesthesia monitoring is to continuously observe the patient's physiological condition and maintain appropriate levels of anesthesia throughout the surgical process. These systems assist anesthesiologists in tracking vital parameters and making timely adjustments to anesthetic delivery.

During surgical operations, anesthetic drugs are administered to induce unconsciousness, analgesia, and muscle relaxation. However, the human body responds differently to anesthetic agents depending on factors such as age, weight, medical history, and type of surgery.[1] Therefore, continuous monitoring is required to maintain the appropriate balance between adequate anesthesia and patient safety.

Traditional anesthesia monitoring systems typically consist of a combination of physiological monitoring devices and gas delivery equipment. These systems measure vital parameters such as heart rate, blood pressure, oxygen saturation (SpO_2), respiratory rate, body temperature, and carbon dioxide concentration. Monitoring these parameters helps clinicians evaluate the patient’s cardiovascular, respiratory, and metabolic conditions during surgery.

In most operating rooms, anesthesia workstations integrate several monitoring modules that display real-time patient data.[2] These monitoring devices provide visual and audible alarms when parameters exceed predefined safety limits. The anesthesiologist continuously observes the monitoring system and manually adjusts anesthetic drug dosage or gas flow when abnormal conditions occur.

Another important component of anesthesia monitoring systems is the anesthetic gas delivery mechanism. This subsystem regulates the mixture of oxygen and anesthetic

gases supplied to the patient. Mechanical flow meters, vaporizers, and breathing circuits are commonly used to control the concentration of anesthetic gases. [3]Accurate gas delivery is essential because excessive anesthetic concentration may lead to respiratory depression, while insufficient anesthesia can result in patient awareness during surgery.

FIG 2.1 Anesthesia Workstation with Patient Monitoring Equipment

Despite the availability of modern anesthesia machines, many conventional monitoring systems still rely heavily on manual observation and decision-making by medical professionals.[4] Continuous human supervision is required to interpret physiological signals and respond to changes in patient condition. This dependence on manual monitoring increases the possibility of delayed response in emergency situations.

Additionally, many existing anesthesia monitoring systems operate as standalone units without advanced data integration or remote monitoring capabilities.[5]Limited connectivity restricts the ability to analyze patient data over extended periods or provide real-time alerts to medical personnel outside the operating room.

Another challenge is the high cost and complexity of advanced anesthesia workstations, which can limit their availability in smaller hospitals and resource-limited healthcare environments. As a result, there is a growing need for more

intelligent, cost-effective, and automated monitoring solutions that can enhance patient safety while reducing the workload on anesthesiologists.

These limitations have motivated researchers to explore new approaches that integrate modern technologies such as embedded systems, biomedical sensors, and intelligent monitoring techniques to improve the efficiency and reliability of anesthesia monitoring systems.

Physiological monitoring sensors play a crucial role in anesthesia monitoring systems by measuring vital signs that reflect the patient's health condition during surgery. [6]These sensors convert biological signals into electrical signals that can be processed and displayed by monitoring devices.

Heart rate monitoring sensors are also widely used in anesthesia systems.[7] These sensors detect changes in blood flow or electrical signals generated by the heart and calculate the number of heartbeats per minute. Monitoring heart rate helps clinicians detect cardiovascular abnormalities during anesthesia.

Temperature sensors are used to monitor body temperature during surgical procedures. Anesthesia can affect the body's natural temperature regulation mechanisms, making temperature monitoring important to prevent conditions such as hypothermia.

Respiratory monitoring sensors measure breathing patterns and respiratory rate. These sensors help ensure that the patient is breathing properly and that oxygen delivery is adequate. Monitoring respiratory parameters is particularly important when patients are under general anesthesia because their natural breathing may be suppressed.

The integration of various physiological sensors allows anesthesia monitoring systems to provide a comprehensive view of the patient's physiological condition during surgery.

Gas monitoring systems are an important component of anesthesia machines because they measure the concentration of gases delivered to the patient. During anesthesia, a mixture of oxygen and anesthetic gases is administered through a

breathing circuit. Accurate measurement of gas concentration is essential to maintain the correct depth of anesthesia.

These sensors detect the presence and concentration of specific gases and convert them into electrical signals that can be processed by monitoring systems.

2.2 IDENTIFIED GAPS

Several research studies have explored the development of monitoring systems for medical and healthcare applications.[8] Researchers have proposed various approaches to improve patient monitoring using advanced sensors, embedded systems, and wireless communication technologies.

Early monitoring systems primarily focused on measuring individual physiological parameters such as heart rate, oxygen saturation, and body temperature. These systems provided basic monitoring capabilities but lacked integration with advanced communication technologies.

Recent research has focused on developing smart healthcare monitoring systems that integrate multiple sensors and embedded processors.[9] These systems aim to provide continuous monitoring of patient parameters and improve the efficiency of medical services.

Some studies have also investigated the use of wireless communication technologies for transmitting patient data to remote monitoring stations.[10]Wireless monitoring systems enable healthcare professionals to monitor patients from distant locations and provide timely medical assistance.

1. Dependence on Manual Monitoring

Most existing systems require continuous human supervision to interpret data and make decisions. This increases workload and introduces the risk of delayed response during emergency conditions.

2. Limited Remote Monitoring Capability

Traditional systems are primarily designed for local monitoring within operating rooms. They lack real-time remote access, making it difficult for medical professionals to monitor patients from distant locations.

3. Lack of Integrated Data Communication

Many systems operate as standalone units without proper data integration or cloud connectivity. This limits long-term data storage, analysis, and real-time alert sharing.

4. High Cost and Complexity

Advanced anesthesia workstations are expensive and complex, making them inaccessible to smaller hospitals and resource-limited healthcare settings.

5. Limited Automation and Intelligence

Existing systems mainly rely on threshold-based alarms and do not provide intelligent decision support or automated control of anesthetic gas delivery.

6. Lack of Scalable and Modular Design

Conventional systems are not easily scalable or adaptable for different environments such as training systems or low-cost healthcare setups.

2.3 OBJECTIVES OF THE THESIS

The main objective of this thesis is to design and develop a smart, cost-effective, and efficient anesthesia monitoring system using embedded and IoT technologies. The specific objectives are:

Primary Objectives

1. To develop an ESP32-based embedded system for real-time anesthesia monitoring

2. To measure physiological parameters such as SpO_2, heart rate, and body temperature

3. To monitor environmental parameters including gas concentration, pressure, and flow rate

4. To implement real-time alert mechanisms using buzzer and visual indicators
5. To design an assistive gas flow control mechanism using a solenoid valve

Secondary Objectives

- To integrate IoT for remote monitoring and data transmission
- To reduce dependency on manual supervision and improve response time
- To design a low-cost and modular system suitable for resource-constrained environments
- To provide a scalable platform for future enhancements such as intelligent control systems

The literature review highlights the importance of anesthesia monitoring systems in ensuring patient safety during surgical procedures. It examines existing technologies, including physiological sensors, gas monitoring systems, and IoT-based healthcare solutions. The review also identifies the limitations of traditional systems, such as lack of automation, limited connectivity, and high cost. Recent advancements in embedded systems and IoT have shown potential to overcome these challenges by enabling real-time monitoring.

CHAPTER 3

METHODOLOGY/ SYSTEM DESIGN

3.1 INTERNET OF THINGS

The Internet of Things (IoT) is a technology that enables physical devices to connect, communicate, and exchange data over the internet. It integrates sensors, microcontrollers, and communication networks to collect and transmit real-time information. IoT systems are widely used in various fields such as healthcare, agriculture, smart homes, and industrial automation. In healthcare, IoT plays a vital role in monitoring patient conditions remotely and continuously. Devices equipped with sensors can measure physiological parameters and send the data to cloud platforms for analysis and storage. This allows doctors and medical staff to access patient information from anywhere, improving response time and decision-making. IoT also supports automation by enabling systems to take actions based on predefined conditions. The use of wireless technologies such as Wi-Fi and Bluetooth makes IoT systems flexible and easy to deploy. Additionally, IoT helps in reducing human effort, improving accuracy, and enhancing system efficiency. Overall, IoT is a key technology driving the development of intelligent and connected systems in modern applications.

3.2 IOT IN HEALTHCARE MONITORING

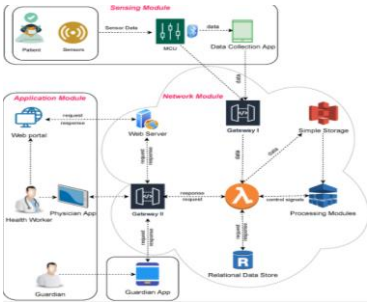


FIG 3.2.1 IoT-Based Healthcare Monitoring System Architecture Diagram

The Internet of Things (IoT) has significantly transformed modern healthcare by enabling intelligent monitoring and data communication between medical devices. IoT-based healthcare systems integrate sensors, microcontrollers, and communication networks to collect and transmit patient data in real time.

In IoT-enabled healthcare systems, biomedical sensors continuously monitor physiological parameters and send the collected data to cloud-based platforms through wireless communication technologies. Healthcare professionals can access patient data remotely using computers or mobile devices, allowing them to monitor patient conditions even when they are not physically present.

In critical medical applications such as anesthesia monitoring, IoT integration can help clinicians monitor multiple parameters simultaneously and receive alerts when patient conditions change. The ability to transmit data in real time also supports telemedicine and remote medical assistance.

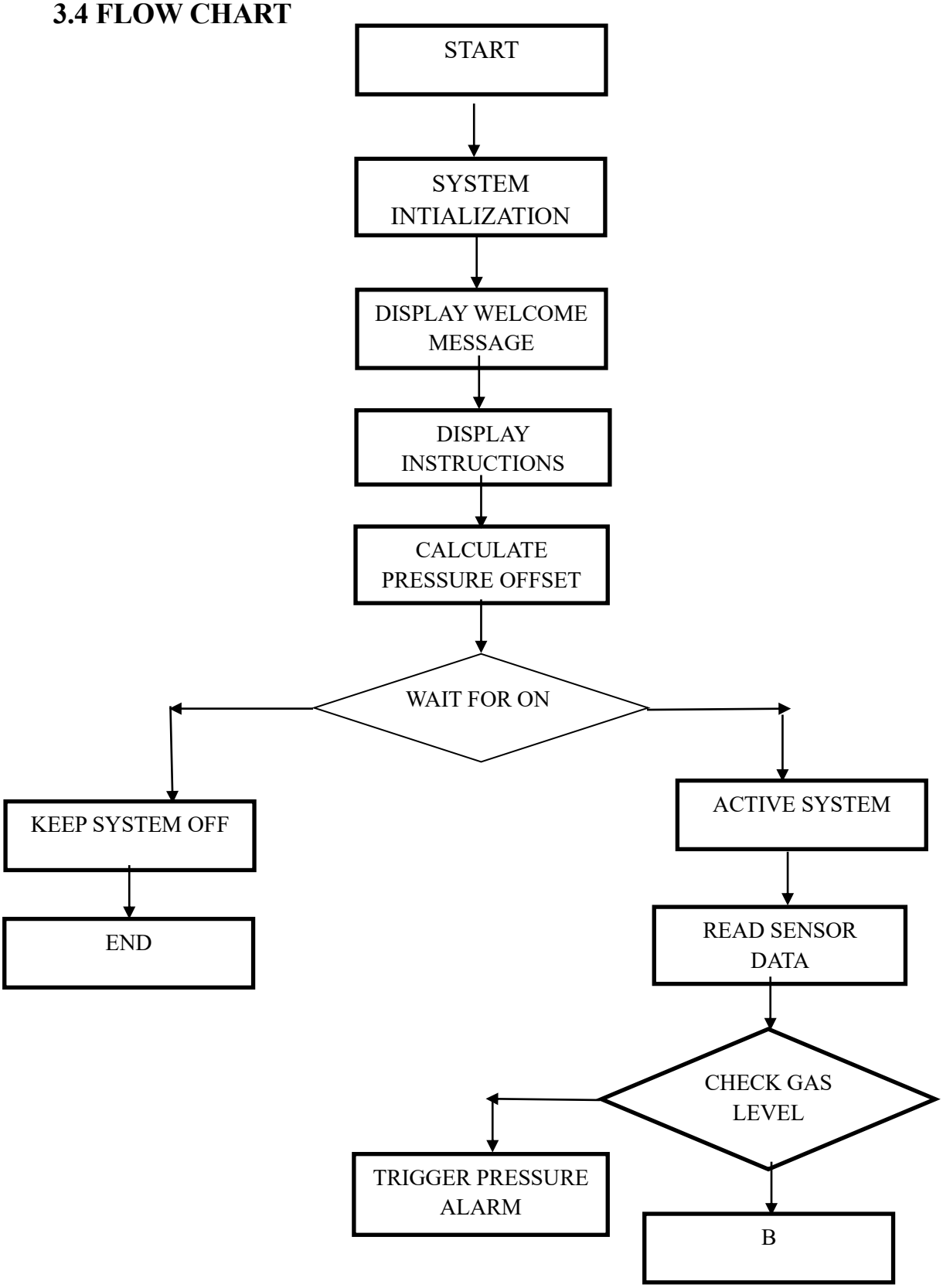
3.3 ESP32 IN MEDICAL AND IOT SYSTEMS

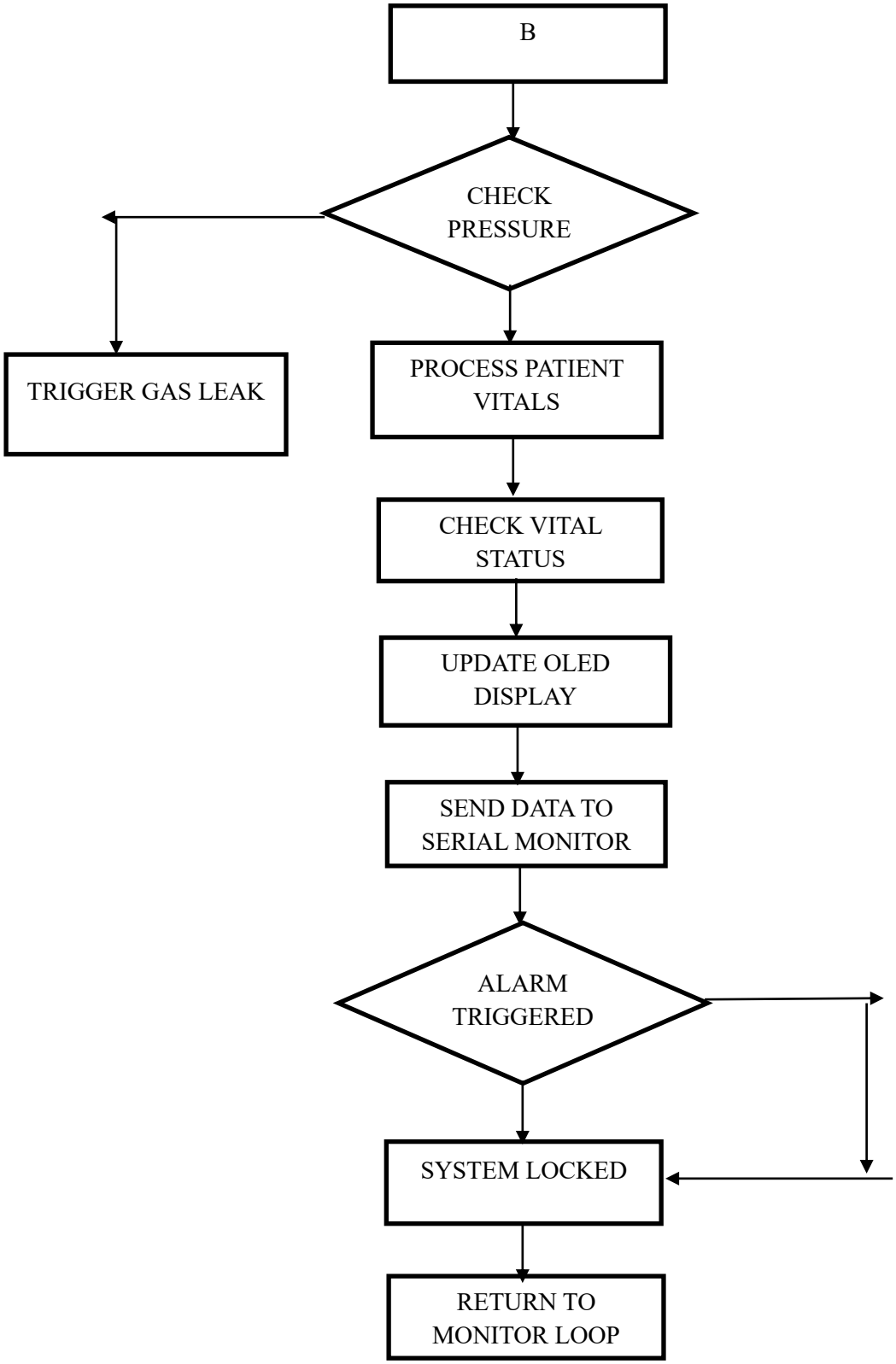
The ESP32 microcontroller has become widely used in IoT-based applications due to its powerful processing capability and integrated wireless communication features. It is a low-cost microcontroller that supports both Wi-Fi and Bluetooth connectivity, making it suitable for various embedded systems and smart monitoring applications.

ESP32 provides multiple input and output interfaces that allow it to connect with different types of sensors and electronic components. These features enable the development of compact and efficient monitoring systems capable of processing and transmitting sensor data.

In medical and healthcare applications, ESP32 can be used to collect physiological data from biomedical sensors and transmit it to cloud platforms for remote monitoring. Its high processing speed allows real-time data processing and quick response to abnormal conditions.The low power consumption and compact design of ESP32 make it suitable for portable and wearable healthcare devices. Because of these advantages, ESP32 has been widely adopted in modern IoT-based healthcare monitoring systems.

3.4 FLOW CHART




B
CHECK PRESSURE
TRIGGER GAS LEAK
PROCESS PATIENT VITALS
CHECK VITAL STATUS
UPDATE OLED DISPLAY
SEND DATA TO SERIAL MONITOR
ALARM TRIGGERED
SYSTEM LOCKED
RETURN TO MONITOR LOOP

3.5 METHODOLOGY

1. **System Initialization**

   The system begins by initializing the ESP32 microcontroller and configuring all input and output pins. Communication protocols such as I²C are established for sensor and display interfacing. The OLED display shows a startup message followed by user instructions.

2. **Sensor Setup and Calibration**

   All sensors including MAX30102, DS18B20, gas sensor, pressure sensor, and flow sensor are initialized. The pressure sensor is calibrated by calculating an offset value to ensure accurate readings.

3. **User Activation Control**

   A push button is used to toggle the system between ON and OFF states. When activated, the system starts monitoring all parameters; otherwise, it remains idle.

4. **Data Acquisition from Sensors**

   The system continuously collects data from all sensors. The MAX30102 sensor provides raw IR and red signals, temperature is obtained from DS18B20, gas levels are read from the MQ-135 sensor, pressure is measured using the pressure sensor, and flow rate is calculated using pulse signals from the flow sensor.

5. **Signal Processing and Parameter Calculation**

   The collected sensor data is processed to obtain meaningful parameters. SpO_2 and heart rate are calculated using the SpO_2 algorithm, while temperature, gas levels, pressure, and flow rate are computed directly.

6. **Safety Monitoring and Threshold Checking**

   All measured parameters are compared with predefined threshold values. Conditions such as gas leakage and high pressure are continuously monitored to ensure safe operation.

7. **Emergency Alert and Control Action**

   If any critical condition is detected, the system triggers an emergency response. The

relay is turned OFF to stop gas flow, a buzzer alert is activated, and warning messages are displayed on the OLED screen.

8. **Vital Status Evaluation**

   The system analyzes heart rate values and classifies them as normal, low, or high. Based on this status, visual indicators such as LEDs are used to represent system condition.

9. **Real-Time Display and Monitoring**

   All parameters including heart rate, SpO_2, temperature, gas concentration, pressure, and system status are displayed on the OLED screen for continuous monitoring.

10. **Data Logging and Serial Output**

    The system sends real-time data to the serial monitor for debugging and analysis. Flow rate calculations are also displayed through serial output.

11. **System Reset and Continuous Operation**

    After an emergency condition, the system automatically resets after a fixed time interval and resumes monitoring. The entire process repeats continuously for real-time operation.

The methodology of the proposed system demonstrates a structured approach for real-time anesthesia monitoring and control. It integrates multiple sensors with the ESP32 microcontroller to continuously acquire and process physiological and environmental data. The system ensures safety through threshold-based monitoring and immediate alert mechanisms. The inclusion of user control, real-time display, and automatic response enhances system reliability and usability. Overall, the methodology provides an efficient framework for developing intelligent and assistive healthcare monitoring systems.

CHAPTER 4

HARDWARE COMPONENTS

4.1 ESP32 DEV MODULE

The ESP32 Dev Module (ESP32-WROOM-32) is a high-performance microcontroller board designed for embedded and Internet of Things (IoT) applications. It is built around the ESP32 chip developed by Espressif Systems. The module integrates a dual-core 32-bit processor, built-in Wi-Fi, and Bluetooth communication capabilities, allowing the device to process data and communicate wirelessly with other systems.

One of the main advantages of the ESP32 module is that it combines processing capability, wireless communication, and multiple input/output interfaces in a single compact board. It supports digital and analog pins that allow easy interfacing with various sensors, displays, and actuators. Because of these features, the ESP32 Dev Module is widely used in smart monitoring systems, healthcare devices, automation systems, and IoT-based control applications.

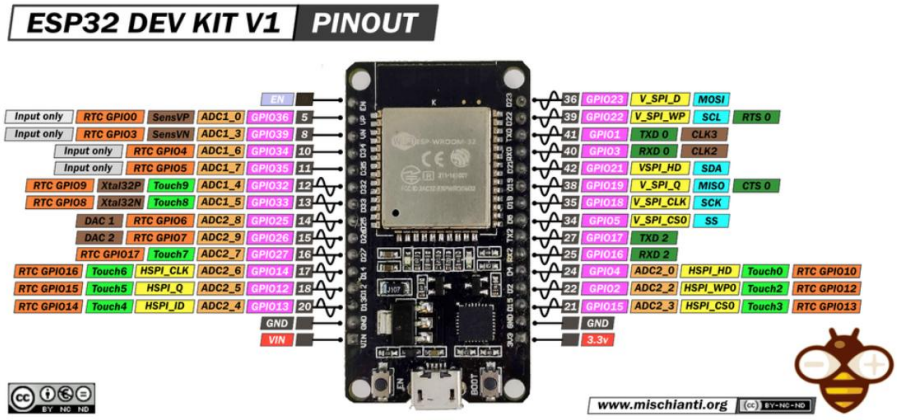


FIG 4.1 ESP32 Dev Kit V1 Pin Configuration Diagram

Role in the Proposed System

In this project, the ESP32 Dev Module acts as the central controller of the anesthesia monitoring system. It is responsible for collecting data from multiple sensors, processing that information, and controlling the output devices based on the system conditions.

The ESP32 receives input data from sensors such as:

- SpO_2 sensor – monitors the patient's blood oxygen level.
- Temperature sensor – measures the patient's body temperature.
- Gas sensor – detects the concentration of anesthetic gases.
- Pressure sensor – monitors the pressure of the gas supply.
- Flow sensor – measures the flow rate of anesthetic gases.

After acquiring the sensor data, the ESP32 processes the information and determines whether the parameters are within safe limits. The processed data is then displayed on the OLED display to provide real-time monitoring of patient conditions.

If any abnormal condition is detected, the ESP32 activates safety mechanisms such as:

- Buzzer – to generate an alarm alert.
- Solenoid valve – to regulate or stop the flow of anesthetic gases when required.

The ESP32 Dev Module is used during real-time monitoring and control of the anesthesia system. It continuously reads sensor values, processes the information, and ensures that the system operates within safe medical limits. Because of its high processing speed and wireless capability, it can also support remote monitoring and IoT-based healthcare applications, allowing medical staff to observe patient data from external devices if required.

Why It Is Suitable for This Project

The ESP32 is suitable for this system because it offers:

- High processing speed with a dual-core processor
- Built-in Wi-Fi and Bluetooth for IoT connectivity
- Multiple GPIO pins for connecting sensors and actuators

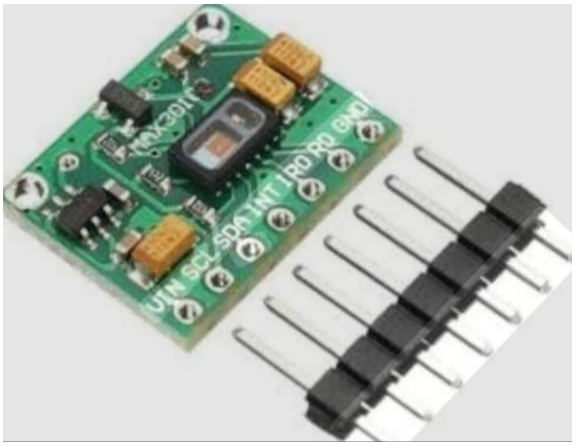
- Low power consumption
- Compact size suitable for embedded medical devices

These features make the ESP32 Dev Module an efficient and reliable controller for the proposed anesthesia monitoring system.

4.2 MAX30102 PULSE OXIMETER AND HEART RATE SENSOR

The MAX30102 Pulse Oximeter and Heart Rate Sensor Module is a compact biomedical sensor used to measure blood oxygen saturation (SpO_2) and heart rate. The module is developed by Analog Devices (originally part of Maxim Integrated) and is widely used in wearable health devices and medical monitoring systems.

The MAX30102 operates using the principle of photoplethysmography (PPG). It contains two light-emitting diodes (LEDs), typically red and infrared, along with a highly sensitive photodetector. When a finger is placed over the sensor, the LEDs emit light through the skin and blood vessels. The photodetector measures the amount of light absorbed by the blood. Since oxygenated and deoxygenated blood absorb light differently, the module can calculate the percentage of oxygen present in the blood as well as determine the heart rate based on changes in blood flow.



4.2 MAX30102 Pulse Oximeter and Heart Rate Sensor

The sensor communicates with microcontrollers through the I²C communication interface, which allows easy integration with development boards such as the ESP32.

Role in the Proposed System

In the proposed anesthesia monitoring system, the MAX30102 sensor plays a crucial role in monitoring the patient's vital physiological parameters. The sensor continuously measures the patient's blood oxygen saturation (SpO_2) and heart rate during the surgical procedure.

The sensor is connected to the ESP32 microcontroller, which collects the data from the MAX30102 through the I²C interface. The ESP32 then processes this information and displays the measured values on the OLED display for real-time monitoring by medical staff.

If the oxygen saturation level or heart rate falls outside the safe range, the system can generate an alert through the buzzer to notify the medical personnel. This helps in taking immediate action to maintain the patient's safety during anesthesia administration**.**

The MAX30102 sensor is used throughout the anesthesia process to continuously monitor the patient's blood oxygen level and heart rate. These parameters are critical indicators of the patient's physiological condition during surgery. Continuous monitoring ensures that any sudden drop in oxygen saturation or abnormal heart rate can be detected early, allowing prompt corrective measures.

Why It Is Suitable for This Project

The MAX30102 sensor is suitable for this project due to several advantages:

- Provides accurate measurement of SpO_2 and heart rate
- Uses non-invasive optical sensing technology
- Compact and lightweight design suitable for embedded medical devices
- Supports I²C communication, making integration with ESP32 simple
- Low power consumption, which is ideal for continuous monitoring systems.

4.3 DS18B20 DIGITAL TEMPERATURE SENSOR

The DS18B20 Digital Temperature Sensor is a widely used electronic sensor designed to measure temperature with high accuracy. It is developed by Analog Devices (formerly Maxim Integrated). The sensor provides digital temperature readings and communicates with microcontrollers using the 1-Wire communication protocol, which requires only a single data line for communication.

FIG 4.3 DS18B20 Digital Temperature Sensor

The DS18B20 sensor can measure temperatures in the range of −55°C to +125°C, with an accuracy of approximately ±0.5°C within the typical operating range. Unlike analog temperature sensors, the DS18B20 converts temperature directly into a digital signal, which reduces noise and improves measurement reliability.

The sensor is available in different forms such as a TO-92 package or a waterproof probe, making it suitable for various environmental and biomedical monitoring applications. Due to its simple interface and reliable performance, it is commonly used in embedded systems, industrial monitoring, and medical-related projects**.**

Role in the Proposed System

In the proposed anesthesia monitoring system, the DS18B20 temperature sensor is used to monitor the patient's body temperature. Maintaining a stable body temperature during surgery is important because anesthesia can sometimes cause changes in the patient's thermal regulation.

The sensor continuously measures the temperature and sends the data to the ESP32 microcontroller through the 1-Wire interface. The ESP32 processes this information and displays the measured temperature on the OLED display so that medical staff can monitor it in real time.

If the temperature value moves outside the safe range, the system can trigger alerts using the buzzer, helping the medical team take immediate action**.**

The DS18B20 sensor is used during the entire surgical procedure to continuously monitor the patient's body temperature. Continuous monitoring helps detect conditions such as hypothermia or abnormal temperature rise, which may occur during anesthesia or surgical procedures.

Why It Is Suitable for This Project

The DS18B20 temperature sensor is suitable for this project because of the following features:

- Provides accurate digital temperature measurements
- Uses 1-Wire communication, requiring only one data pin
- Wide temperature measurement range
- High reliability and stability in continuous monitoring applications
- Easy integration with microcontrollers such as the ESP32

4.4 MQ-135 GAS SENSOR MODULE

The MQ-135 Gas Sensor Module is an air quality and gas detection sensor commonly used to detect harmful gases in the environment. It is manufactured by Hanwei Electronics and is widely used in environmental monitoring, industrial safety systems, and gas detection applications.

The MQ-135 sensor operates based on a metal oxide semiconductor (MOS) sensing element. When certain gases are present in the air, the electrical resistance of the sensing material changes. This change in resistance is converted into an electrical signal that can be measured by a microcontroller. The sensor is capable of detecting

gases such as ammonia (NH_3), nitrogen oxides (NOx), benzene, smoke, carbon dioxide (CO_2), and other harmful gases.

The MQ-135 module typically provides both analog and digital outputs, allowing it to be easily interfaced with microcontrollers like the ESP32. The analog output gives a proportional signal representing gas concentration, while the digital output can be used for threshold-based gas detection.

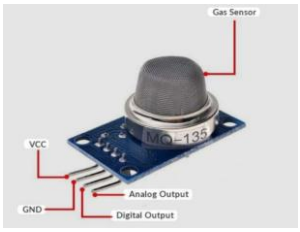


FIG 4.4 MQ-135 Gas Sensor Module

Role in the Proposed System

In the proposed anesthesia monitoring system, the MQ-135 gas sensor is used to monitor the presence and concentration of anesthetic gases or harmful gases in the surrounding environment. Monitoring gas levels is important because leakage or excessive accumulation of anesthetic gases can be dangerous for both the patient and medical staff.

The sensor detects gas concentration in the environment and sends the corresponding signal to the ESP32 microcontroller. The ESP32 processes this data and displays the gas level on the OLED display for real-time monitoring.

If the gas concentration exceeds a safe threshold level, the ESP32 activates the buzzer alarm and can also trigger the solenoid valve to regulate or stop the gas supply. This helps maintain a safe operating environment in the anesthesia system.

The MQ-135 gas sensor is used continuously during the operation of the anesthesia monitoring system to detect the presence of anesthetic gases or any harmful gas leakage. Continuous monitoring ensures that abnormal gas levels can be identified

immediately, allowing corrective action to be taken before the situation becomes hazardous.

Why It Is Suitable for This Project

The MQ-135 gas sensor is suitable for this project because of the following advantages:

- Ability to detect multiple harmful gases.
- Provides both analog and digital outputs.
- Suitable for air quality and gas leakage monitoring.
- Easy integration with microcontrollers such as the ESP32.
- Reliable and widely used in safety monitoring systems.

4.5 YF-S401 FLOW SENSOR MODULE

The YF-S401 Flow Sensor Module is a compact sensor designed to measure the flow rate of liquids or gases passing through a pipe or tube. The sensor works based on a Hall-effect sensing principle. Inside the sensor, a small turbine rotates when fluid flows through it. As the turbine spins, a magnet attached to it passes a Hall-effect sensor, generating electrical pulses.

FIG 4.5 YF-S401 Flow Sensor Module

The frequency of these pulses is directly proportional to the flow rate of the fluid. A microcontroller can count these pulses to determine the flow rate and total volume of the fluid passing through the sensor. The YF-S401 sensor typically provides a digital pulse output, making it easy to interface with microcontrollers such as the ESP32.

Because of its simple working mechanism and reliable measurement capability, the YF-S401 flow sensor is widely used in fluid monitoring systems, medical devices, industrial automation, and smart control systems.

Role in the Proposed System

In the proposed anesthesia monitoring system, the YF-S401 flow sensor is used to monitor the flow rate of anesthetic gases delivered to the patient. Maintaining the correct flow rate of anesthesia is critical because too much or too little gas can affect the patient's safety during surgery.

The sensor measures the flow of gas in the anesthesia delivery line and sends pulse signals to the ESP32 microcontroller. The ESP32 counts these pulses and calculates the flow rate of the gas. This information is then displayed on the OLED display, allowing medical staff to monitor the flow rate in real time.

The YF-S401 flow sensor is used during the continuous operation of the anesthesia delivery system to measure and monitor the flow rate of gases being supplied to the patient. Continuous monitoring ensures that the correct amount of anesthetic gas is delivered throughout the surgical procedure.

Why It Is Suitable for This Project

The YF-S401 flow sensor is suitable for this project because of the following features:

- Provides accurate flow rate measurement
- Uses Hall-effect sensing technology
- Produces digital pulse output, which is easy to interface with ESP32
- Compact and lightweight design suitable for embedded systems
- Reliable performance for continuous monitoring applications

4.6 HX710B AIR PRESSURE SENSOR MODULE (0–40 KPA)

The HX710B Air Pressure Sensor Module (0–40 kPa) is an electronic sensor module designed to measure low air pressure levels with high accuracy. It consists of a pressure sensing element and the HX710B analog-to-digital converter (ADC), which converts the analog pressure signal into a digital output that can be read by a microcontroller.

The sensor measures pressure changes within a range of 0 to 40 kilopascals (kPa), making it suitable for applications that require monitoring of low-pressure gas or air systems. The module communicates with microcontrollers through a digital interface, allowing accurate pressure measurements without the need for complex signal conditioning circuits.

FIG 4.6 HX710B Air Pressure Sensor Module (0–40 kPa)

Role in the Proposed System

In the proposed anesthesia monitoring system, the HX710B air pressure sensor is used to monitor the pressure of anesthetic gases in the delivery system. Maintaining proper gas pressure is essential for ensuring that anesthesia is delivered safely and consistently to the patient during surgery.

The sensor continuously measures the pressure in the gas supply line and sends the pressure data to the ESP32 microcontroller. The ESP32 processes this information and displays the pressure values on the OLED display for real-time monitoring.

If the pressure level becomes too high or too low, the system can trigger a buzzer alarm to alert the medical staff. Additionally, the ESP32 can control the solenoid valve to regulate the gas supply and maintain the required pressure level.

The HX710B pressure sensor is used during the continuous operation of the anesthesia monitoring system to track the pressure of the anesthetic gas supply. Continuous pressure monitoring is important because unstable gas pressure can lead to improper delivery of anesthesia.

Why It Is Suitable for This Project

The HX710B air pressure sensor module is suitable for this project due to the following advantages:

- Capable of measuring low air pressure accurately (0–40 kPa range)
- Provides digital output, reducing signal noise and improving accuracy
- Easy integration with microcontrollers such as the ESP32
- Compact design suitable for embedded monitoring systems
- Reliable for continuous pressure monitoring applications

4.7 12V DC NORMALLY CLOSED SOLENOID VALVE

The 12V DC Normally Closed (NC) Solenoid Valve is an electromechanical device used to control the flow of liquids or gases in a system. It operates using an electromagnetic coil that opens or closes the valve when electrical power is applied.

In a normally closed solenoid valve, the valve remains closed when no power is supplied. When a 12V DC electrical signal is applied to the coil, an electromagnetic field is generated, which pulls a plunger inside the valve and opens the flow path. Once the power is removed, a spring mechanism pushes the plunger back to its original position, automatically closing the valve.

FIG 4.7 12V DC Normally Closed Solenoid Valve

Solenoid valves are widely used in automation systems, industrial fluid control, medical gas delivery systems, irrigation systems, and safety control mechanisms because they allow fast and reliable control of fluid or gas flow.

Role in the Proposed System

In the proposed anesthesia monitoring system, the 12V DC normally closed solenoid valve is used to control the flow of anesthetic gases supplied to the patient.

The valve is connected to the gas delivery line, and its operation is controlled by the ESP32 microcontroller. Based on the sensor readings from the gas sensor, pressure sensor, and flow sensor, the ESP32 determines whether the gas flow is within the safe operating range.

- If the system detects normal operating conditions, the solenoid valve allows controlled gas flow.
- If an abnormal condition is detected, such as excessive gas concentration, abnormal pressure, or unsafe flow rate, the ESP32 can immediately close the solenoid valve to stop the gas supply.

This mechanism helps prevent potential hazards and improves patient safety during anesthesia delivery.

The solenoid valve is used during the operation of the anesthesia delivery system to regulate and control the flow of anesthetic gases. It remains closed by default and opens only when the system permits gas flow.

If the monitoring system detects unsafe conditions, the valve can quickly close to stop or regulate the gas supply, thereby preventing complications during the surgical procedure.

Why It Is Suitable for This Project

The 12V DC normally closed solenoid valve is suitable for this project due to the following features:

- Provides fast and automatic control of gas flow
- Normally closed design improves safety, as the valve shuts when power is lost
- Operates using a 12V DC supply, which is common in embedded control systems

4.8 5V SINGLE-CHANNEL RELAY MODULE

The 5V Single-Channel Relay Module (Opto-Isolated) is an electronic switching device used to control high-power electrical components using a low-voltage microcontroller signal. A relay works as an electromagnetic switch, allowing a small control signal to safely operate devices that require higher voltage or current.

The module operates using a 5V supply and typically contains a relay switch, a driver transistor circuit, and an optocoupler (optical isolator). The optocoupler provides electrical isolation between the microcontroller circuit and the high-power switching circuit. This isolation helps protect sensitive electronic components such as microcontrollers from voltage spikes or electrical noise.

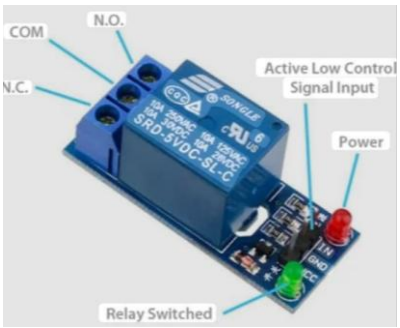


FIG 4.8 Single-Channel Relay Module (Opto-Isolated)

The relay module usually includes three switching terminals:

- COM (Common)
- NO (Normally Open)
- NC (Normally Closed)

By controlling the input signal from the microcontroller, the relay can connect or disconnect the electrical path between these terminals. Relay modules are widely used in automation systems, smart control systems, industrial equipment, and embedded electronics projects.

Role in the Proposed System

In the proposed anesthesia monitoring system, the 5V single-channel relay module is used as an interface between the ESP32 microcontroller and high-power components, such as the 12V solenoid valve.

Since the ESP32 operates at a low voltage and cannot directly drive higher-power devices, the relay module acts as a switching interface. When the ESP32 detects abnormal conditions from sensors such as gas concentration, pressure, or flow rate, it sends a control signal to the relay module.

The relay then activates or deactivates the connected device, such as controlling the solenoid valve, which regulates the flow of anesthetic gases. This ensures that the microcontroller can safely control external components without being exposed to higher voltages.

The relay module is used whenever the system needs to switch external devices on or off based on sensor readings. During the operation of the anesthesia monitoring system, the ESP32 continuously evaluates data from all sensors.

The 5V Single-Channel Relay Module (Opto-Isolated) is suitable for this project due to the following features:

- Allows low-voltage microcontrollers to control high-power devices

- Opto-isolation protects the ESP32 from electrical noise and voltage spikes
- Supports switching of AC or DC loads

4.9 0.96″ OLED DISPLAY MODULE (SSD1306)

The 0.96″ OLED Display Module (SSD1306) is a compact graphical display used in embedded systems to present real-time information. It is based on the SSD1306 display driver controller, developed by Solomon Systech. The display typically has a resolution of 128 × 64 pixels, allowing it to show text, numbers, and simple graphics clearly.

OLED stands for Organic Light Emitting Diode, a technology in which each pixel emits its own light. Unlike traditional LCD displays, OLED screens do not require a backlight, which results in better contrast, lower power consumption, and thinner display modules**.**

The module commonly communicates with microcontrollers using the I²C communication protocol, requiring only two main data lines: SDA (data) and SCL (clock). This makes it easy to integrate with microcontrollers such as the ESP32.

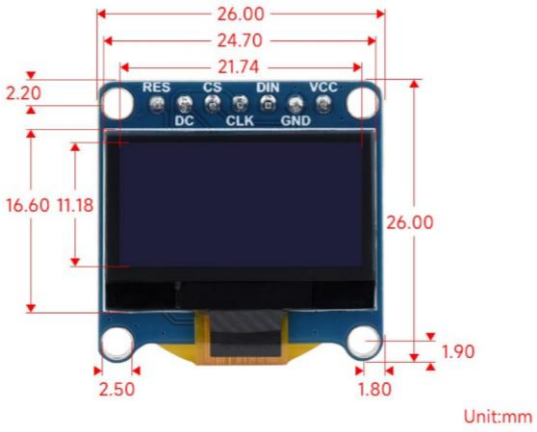


FIG 4.9 0.96″ OLED Display Module

Because of its small size, low power consumption, and clear visibility, the SSD1306 OLED display is widely used in IoT devices, wearable electronics, medical monitoring systems, and embedded control systems.

Role in the Proposed System

In the proposed anesthesia monitoring system, the 0.96″ OLED display module is used to display real-time sensor data and system status.

The ESP32 microcontroller collects data from multiple sensors, including the SpO_2 sensor, temperature sensor, gas sensor, pressure sensor, and flow sensor. After processing the sensor readings, the ESP32 sends the information to the OLED display**.**

The OLED screen presents important parameters such as:

- Blood oxygen level (SpO_2)
- Heart rate
- Body temperature
- Gas concentration
- Gas pressure
- Gas flow rate

Displaying this information on the OLED screen allows medical personnel to continuously monitor the patient's physiological parameters and system conditions during anesthesia administration.

The OLED display is used throughout the operation of the anesthesia monitoring system. It continuously shows updated sensor values and system alerts in real time.

Whenever the system detects abnormal conditions, warning messages or alert information can also be displayed on the screen. This helps medical staff quickly understand the system status and respond appropriately.

Why It Is Suitable for This Project

The 0.96″ OLED Display Module (SSD1306) is suitable for this project because of the following features:

- Provides clear and high-contrast display
- Low power consumption, suitable for embedded systems

- Supports I²C communication, requiring fewer microcontroller pins.

4.10 ACTIVE BUZZER MODULE

The Active Buzzer Module is an electronic sound-producing device used in embedded systems to generate audible alerts or warning signals. Unlike a passive buzzer, an active buzzer contains an internal oscillator circuit, which allows it to produce sound when a voltage is applied. This means the buzzer can generate a tone simply by supplying power, without requiring additional signal generation from the microcontroller**.**

Active buzzer modules typically operate at 3.3V or 5V, making them compatible with common microcontrollers such as the ESP32, Arduino, and other embedded development boards. The module usually consists of a buzzer element and a simple driver circuit mounted on a small printed circuit board with VCC, GND, and signal pins.

FIG 4.10 Active Buzzer Module

Role in the Proposed System

In the proposed anesthesia monitoring system, the active buzzer module is used to provide audio alerts when abnormal conditions are detected. The buzzer is connected to the ESP32 microcontroller, which continuously monitors data from various sensors such as the SpO_2 sensor, temperature sensor, gas sensor, pressure sensor, and flow sensor.When the ESP32 detects that any parameter exceeds the predefined safe limits, it sends a signal to the buzzer module. The buzzer then produces a sound to alert the medical staff about the potential issue.

4.11 LM2596 DC-DC BUCK CONVERTER MODULE

The LM2596 DC-DC Buck Converter Module is a voltage regulation module used to step down (reduce) a higher DC voltage to a lower DC voltage efficiently. It is based on the LM2596 switching regulator integrated circuit, manufactured by Texas Instruments.

Unlike linear voltage regulators, which dissipate excess voltage as heat, the LM2596 module uses switching regulation technology. This method converts voltage with high efficiency, typically up to 90%, making it suitable for powering electronic circuits that require stable voltage levels.

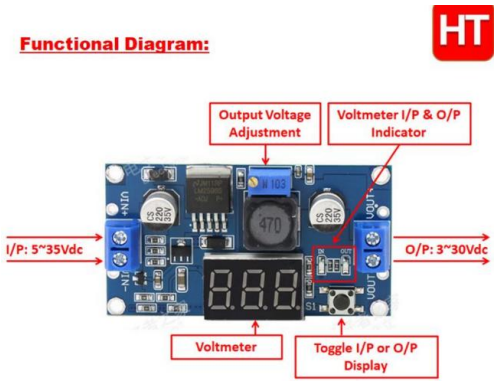


FIG 4.11 LM2596 DC-DC Buck Converter Module

Because of its high efficiency, compact size, and adjustable output capability, the LM2596 buck converter module is widely used in embedded systems, power supply circuits, battery-powered devices, IoT systems, and electronic control applications.

Role in the Proposed System

In the proposed anesthesia monitoring system, the LM2596 buck converter module is used to provide a stable and regulated power supply to different electronic components.

Some components in the system operate at different voltage levels. For example:

- The ESP32 microcontroller typically requires 3.3V or 5V.

- The OLED display and sensors require stable low-voltage power.
- The system power source may provide a higher input voltage, such as 9V or 12V.

The LM2596 module converts the higher input voltage into the required lower voltage levels to safely power the ESP32 and other electronic components. This ensures stable system operation and protects sensitive components from voltage fluctuations.

The LM2596 buck converter module is used whenever the system is powered on. It continuously regulates the input voltage and supplies the required stable voltage to the microcontroller, sensors, and other modules.

Proper voltage regulation is important because unstable power supply can lead to incorrect sensor readings, microcontroller malfunction, or system failure.

Why It Is Suitable for This Project

The LM2596 DC-DC Buck Converter Module is suitable for this project because of the following features:

- Provides efficient voltage step-down conversion
- High efficiency compared to linear regulators

These features make the LM2596 module an important component for providing reliable power management in the proposed anesthesia monitoring system.

4.12 12V DC POWER ADAPTER

The 12V DC Power Adapter is an external power supply device used to convert AC mains electricity into a stable 12V DC output suitable for electronic circuits and embedded systems. It typically plugs into a standard electrical outlet and provides a regulated DC voltage through a cable with a barrel connector or terminal output.13. 5V DC Power Adapter. The 12V DC power adapter is used whenever the anesthesia monitoring system is powered on. It continuously supplies power to the system during

operation, ensuring that the sensors, microcontroller, display, and control devices function correctly**.**

Why It Is Suitable for This Project

The 12V DC Power Adapter is suitable for this project due to the following reasons:

- Provides stable DC power supply for the system
- Compatible with components such as solenoid valves and relay modules that operate at 12V

These are the Hardware components that are used in the project.

CHAPTER 5

SOFTWARE COMPONENTS

5.1 ARDUINO UNO

Arduino is an open-source hardware and software platform designed for building electronic systems and embedded applications easily. The Arduino software platform mainly consists of the Arduino Integrated Development Environment (IDE) and a collection of libraries that allow users to program microcontroller boards such as Arduino Uno, Mega, Nano, and ESP-based boards.

The platform simplifies embedded system development by providing a user-friendly programming interface, built-in libraries, and straightforward methods to upload programs to hardware boards. Because of this simplicity, Arduino is widely used in IoT systems, robotics, sensor monitoring, medical electronics, and automation projects.

Arduino Integrated Development Environment (IDE)

The Arduino IDE is the primary software used to write, compile, and upload programs to Arduino boards. It supports programming using a simplified version of C/C++ and provides tools for debugging and monitoring system outputs.

Main functions of the Arduino IDE include:

- Code Editor: Allows users to write programs known as sketches.
- Compiler: Converts the written program into machine code that the microcontroller can execute.
- Uploader: Transfers the compiled program to the Arduino board through USB or serial communication.

- Serial Monitor: Displays real-time data sent from the microcontroller, useful for debugging and monitoring sensor outputs.

5.2 ARDUINO PROGRAMMING STRUCTURE

Arduino programs follow a simple structure consisting of two essential functions:

1. **setup() function**
   - Executes only once when the microcontroller starts.
   - Used to initialize pins, sensors, communication protocols, and libraries.
2. **loop() function**
   - Runs continuously after setup().
   - Contains the main program logic that repeats as long as the board is powered.
3. 

FIG 5.1 Arduino Software Platform

This structure simplifies embedded programming compared with traditional microcontroller development.

5.3 ARDUINO LIBRARIES

Arduino provides a large number of pre-built libraries that make it easier to interface with sensors, displays, communication modules, and other electronic components. These libraries eliminate the need to write complex low-level code.

Examples include:

- Sensor libraries (temperature, gas, pressure sensors)
- Communication libraries (I2C, SPI, UART, Wi-Fi, Bluetooth)
- Display libraries (LCD, OLED)

These libraries significantly reduce development time and improve system reliability.

Cross-Platform Compatibility

The Arduino software platform is cross-platform, meaning the IDE can run on multiple operating systems such as:

- Windows
- Linux
- macOS

This allows developers and researchers to build embedded systems using different computing environments.

Role of Arduino Software Platform in the Proposed System

In the proposed Anesthesia Monitoring System, the Arduino software platform is used to:

- Program the microcontroller to read data from physiological and gas sensors.
- Process and analyze sensor values in real time.
- Control alarms or indicators when abnormal conditions are detected.
- Transmit patient monitoring data through communication modules (such as Wi-Fi or IoT platforms).
- Display or log the monitored parameters for medical supervision.

The Arduino software environment enables efficient integration of multiple sensors and communication modules, making it suitable for developing real-time medical monitoring systems.

LIBRARIES USED

1. Arduino.h

The Arduino.h library is the core library of the Arduino platform. It provides basic functions required for embedded programming such as pin control, timing functions (delay, millis), and digital/analog input-output operations. This library is automatically included in most Arduino-based programs and forms the foundation for all hardware interactions in the system.

2. Wire.h

The Wire.h library is used for I²C (Inter-Integrated Circuit) communication. It enables communication between the ESP32 microcontroller and peripheral devices such as sensors and display modules using two wires: SDA (data) and SCL (clock). In this project, it is used to interface with the MAX30102 sensor and OLED display.

3. MAX30105.h

The MAX30105 library is used to interface with the MAX30102 pulse oximeter and heart rate sensor. It provides functions to initialize the sensor, read infrared (IR) and redlight values, and manage sensor data acquisition. This library simplifies communication with the sensor over I²C and enables real-time collection of physiological signals.

4. Spo2_algorithm.h

This library implements the algorithm required to calculate blood oxygen saturation (SpO_2) and heart rate from raw sensor data obtained from the MAX30102. It processes the infrared and red light signals and converts them into meaningful physiological parameters.

5. Adafruit_GFX.h

The Adafruit_GFX library is a graphics library used for drawing text, shapes, and images on display modules. It provides basic functions for rendering characters, lines, and graphics. In this project, it is used as a base library for displaying information on the OLED screen.

6. Adafruit_SSD1306.h

The Adafruit_SSD1306 library is specifically designed to control OLED displays based on the SSD1306 driver. It works along with the Adafruit_GFX library to display text and graphical data. In this system, it is used to show real-time values such as heart rate, SpO_2, temperature, pressure, and system status.

7. OneWire.h

The OneWire library is used for communication with devices that use the One-Wire protocol. This protocol requires only a single data line for communication. In this project, it is used to interface with the DS18B20 temperature sensor.

8. DallasTemperature.h

The Dallas Temperature library works in combination with the OneWire library to simplify communication with temperature sensors like DS18B20. It provides functions to request temperature readings and retrieve values in degrees Celsius. This library enables accurate and easy temperature monitoring in the system.

The software implementation plays a crucial role in enabling real-time monitoring and control of the anesthesia system. Using the Arduino IDE and relevant libraries, the ESP32 efficiently processes sensor data and executes decision-making logic. The integration of multiple libraries simplifies communication with sensors, display modules, and control components. The program ensures continuous monitoring, alert generation, and system safety through both hardware and software mechanisms. Overall, the software design provides a reliable and scalable platform for embedded healthcare applications.

CHAPTER 6

IMPLEMENTATION AND RESULT

6.1 SYSTEM IMPLEMENTATION OVERVIEW

The proposed Anesthesia-Based Patient Monitoring System is implemented using an embedded microcontroller integrated with multiple biomedical and safety sensors. The system continuously monitors patient vitals and environmental conditions while ensuring automatic safety intervention during abnormal conditions.

The implementation combines:

- Real-time physiological monitoring
- Gas and pressure safety detection
- Automated alert and shutdown mechanisms
- Visual feedback through OLED display
- Audio-visual alarms

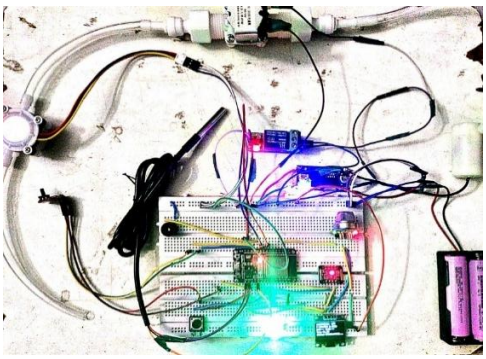


FIG 6.1Anesthesia Monitoring and Assistive Gas Flow Control System

The system operates in two modes:

1. Normal Monitoring Mode
2. Emergency Safety Mode (Alarm Mode)

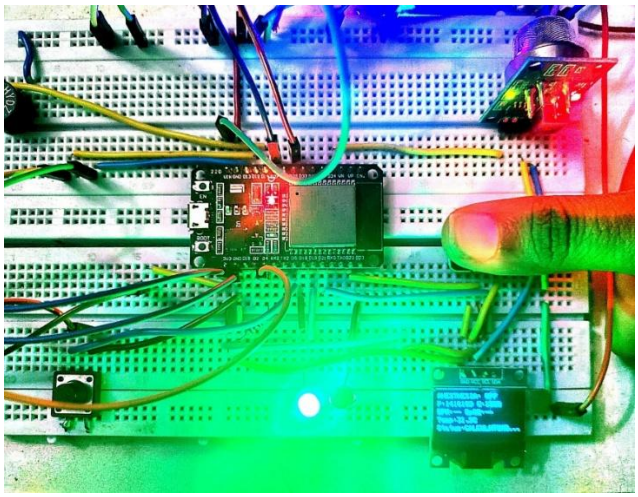


FIG 6.2 Input to the System.

Environmental & Safety Inputs

- Gas Sensor Value (Analog Input)
    - Detects leakage of anesthetic gases
    - Threshold: 2750
- Pressure Sensor (HX711 Interface)
    - Monitors pressure in anesthesia system
    - Maximum safe pressure: 1,800,000 (raw units)
- Flow Sensor (Pulse-based Input)
    - Measures fluid/gas flow rate
    - Used for monitoring (displayed in Serial Monitor)

User Input

- Push Button
    - Used to toggle anesthesia system ON/OFF

- Works only when no alarm is active

System Outputs

The system generates outputs in multiple forms to ensure clear and immediate feedback:

Visual OUT

OLED Displays:

- System status (ON/OFF)
- Pressure and Gas values
- Heart rate (BPM)
- SpO_2 percentage
- Temperature
- Patient status (Stable / Low / High)

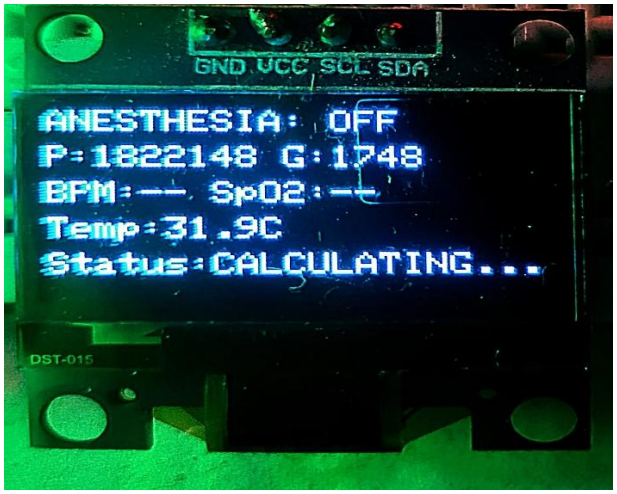


FIG 6.3 Display

Indicator Outputs

- Green LED
  - Indicates normal/stable condition
- Red LED

- Indicates abnormal vitals or emergency

FIG 6.4 ALERT

Audio Output

- Buzzer - Activated during emergency conditions.

Control Output

- Relay (SYSTEM_RELAY)
  - Controls anesthesia delivery
  - Automatically turned OFF during critical conditions

6.2 SYSTEM WORKING

Normal Operation

When the system is activated:

- Sensors begin data acquisition
- OLED displays real-time parameters
- Green LED remains ON

- Relay remains ON (anesthesia active)

Observed Result:

- System successfully tracks real-time vitals
- Stable readings show “STABLE” status

Vital Monitoring Logic

The system evaluates heart rate:

Condition	Heart Rate	Status
Low	< 60 BPM	LOW VITALS
Normal	60–150 BPM	STABLE
High	> 150 BPM	HIGH VITALS

Observed Result:

- Status updates correctly in real time
- LED indication changes accordingly

Gas Leakage Detection

If gas sensor value exceeds threshold:

- Alarm is triggered
- Relay is turned OFF
- Buzzer is activated
- OLED shows alert message

Observed Result:

- Immediate system shutdown prevents hazard

Pressure Monitoring

If pressure exceeds safe limit:

- System triggers emergency alarm
- Anesthesia relay is turned OFF

Observed Result:

- Overpressure condition handled effectively

Emergency Alarm Behavior

When alarm is triggered:

- System enters lock state
- Relay OFF
- Red LED ON
- Buzzer ON
- OLED shows alert

After 20 seconds - System automatically resets

Observed Result:

- Prevents rapid toggling or unsafe restart , Adds safety delay.

The system was successfully implemented and demonstrated stable real-time monitoring and control under test conditions. Sensor integration and data processing performed reliably, ensuring accurate measurements and timely responses. The overall design proved efficient, scalable, and suitable for practical deployment with minimal hardware complexity. Future improvements can focus on enhancing accuracy, robustness, and incorporating advanced features such as IoT-based monitoring for better usability and performance.

CHAPTER 7

DISCUSSION AND CONCLUSION

DISCUSSION

The developed ESP32-based IoT anesthesia monitoring system demonstrates a practical approach toward improving patient safety and reducing manual workload in surgical environments. The system successfully integrates multiple physiological and environmental sensors, including SpO_2, temperature, gas concentration, pressure, and flow sensors, into a unified embedded platform.

One of the key observations from the implementation is the effectiveness of real-time monitoring combined with immediate alert mechanisms. The use of buzzer alerts, LED indicators, and automatic system shutdown ensures that abnormal conditions such as gas leakage, high pressure, or unsafe vital signs are detected and handled promptly. This significantly reduces dependency on continuous manual supervision.

The integration of ESP32 with IoT capabilities allows for remote monitoring and data transmission, which is a major improvement over conventional anesthesia systems that are limited to local monitoring. The system also provides data visualization through OLED display and serial monitoring, enabling easy interpretation of patient parameters.

However, the system still has limitations:

- The gas sensing (MQ-135) is not medically precise for anesthetic gases
- Lack of clinical-grade calibration
- No closed-loop intelligent control (only threshold-based)
- System reliability under real surgical conditions is not validated

CONCLUSION

This project presents a cost-effective and efficient solution for anesthesia monitoring using embedded systems and IoT technology. The proposed system successfully monitors critical physiological and gas-related parameters and provides real-time alerts to ensure patient safety.

By integrating ESP32 with multiple sensors and control mechanisms such as solenoid valves and relays, the system demonstrates the capability to assist in regulating anesthetic gas delivery. The inclusion of IoT connectivity enhances the system by enabling remote monitoring, data logging, and timely response to critical conditions.

Compared to traditional anesthesia monitoring systems, the proposed solution reduces manual effort, improves response time, and offers scalability for future enhancements. Although the system is not intended to replace professional medical equipment, it provides a strong foundation for developing intelligent and affordable healthcare monitoring systems, especially in resource-limited environments.

CHAPTER 8

FUTURE SCOPE

1. Integration of Clinical-Grade Sensors

The current system uses general-purpose sensors suitable for prototyping; however, they may not provide the required accuracy for medical applications. Future enhancements can include the use of certified biomedical sensors for precise measurement of parameters such as oxygen saturation, anesthetic gas concentration, and pressure. This will improve system reliability and make it suitable for clinical environments**.**

2. Advanced Control Algorithms

The existing system operates based on predefined threshold values, which may not be sufficient for handling dynamic patient conditions. Future systems can incorporate advanced control techniques such as PID control, fuzzy logic, or machine learning algorithms to enable adaptive and predictive regulation of anesthetic gas delivery, ensuring smoother and safer operation**.**

3. Expansion of IoT Capabilities

The system can be further improved by integrating cloud-based platforms and mobile applications. This would enable real-time remote monitoring, data storage, and analysis of patient parameters. Features such as automated alerts, notifications, and historical data tracking can assist healthcare professionals in making faster and more informed decisions.

4. Fault Tolerance and System Reliability

For real-world medical applications, system reliability is critical. Future improvements can include redundancy mechanisms such as backup sensors, dual microcontrollers, and fail-safe designs to ensure continuous operation even in case of component failure. This will enhance the safety and robustness of the system**.**

5. Hardware Optimization and Miniaturization

The current prototype can be further developed into a compact and portable device by designing a dedicated PCB and integrating all components into a single unit. Miniaturization will make the system more practical for hospital use and portable healthcare applications.

6. Regulatory Compliance and Clinical Validation

To make the system suitable for real-world deployment, it must undergo proper calibration, testing, and certification according to medical standards. Future work should focus on validating system performance under clinical conditions and ensuring compliance with healthcare safety regulations and standards**.**

7. Integration with Hospital Systems

The system can be extended to integrate with hospital management systems and electronic health records (EHR). This would allow seamless data sharing, centralized monitoring, and improved coordination among medical staff.

Appendices

SOURCE CODE

```
#include <Arduino.h>

#include <Wire.h>

#include "MAX30105.h"

#include "spo2_algorithm.h"

#include <Adafruit_GFX.h>

#include <Adafruit_SSD1306.h>

#include <OneWire.h>

#include <DallasTemperature.h>


// --- PIN DEFINITIONS ---

#define BUTTON_PIN      5

#define SYSTEM_RELAY    25

#define FLOW_SENSOR     27

#define HX_DATA         12

#define HX_CLK          13

#define ONE_WIRE_BUS    4

#define GAS_SENSOR      34

#define BUZZER          14

#define RED_LED         23
```

```
#define GREEN_LED       18


// --- CONSTANTS ---
const long MAX_PRESSURE = 1800000;
const int GAS_THRESHOLD = 2750;
const float FLOW_CAL_FACTOR = 98.0;
#define BUFFER_SIZE 100
// --- OBJECTS ---
OneWire oneWire(ONE_WIRE_BUS);
DallasTemperature sensors(&oneWire);
MAX30105 particleSensor;
Adafruit_SSD1306 display(128, 64, &Wire, -1);
// --- GLOBAL VARIABLES ---
bool systemActive = false;
bool lastButtonState = HIGH;
bool alarmActive = false;
String alarmMessage = "";
String vitalStatus = "STABLE";
unsigned long alarmStartTime = 0;
volatile long pulseCount = 0;
long pressureOffset = 0;
```

```
unsigned long activeStartTime = 0;
uint32_t irBuffer[BUFFER_SIZE];
uint32_t redBuffer[BUFFER_SIZE];
int32_t spo2, heartRate;
int8_t validSPO2, validHeartRate;
void IRAM_ATTR pulseCounter() { pulseCount++; }
long readPressureRaw() {
  unsigned long timeout = millis();
  while (digitalRead(HX_DATA) == HIGH) {
    if (millis() - timeout > 20) return -999;
  }
  unsigned long count = 0;
  for (byte i = 0; i < 24; i++) {
    digitalWrite(HX_CLK, HIGH); delayMicroseconds(1);
    count = count << 1;
    digitalWrite(HX_CLK, LOW); delayMicroseconds(1);
    if (digitalRead(HX_DATA)) count++;
  }
  digitalWrite(HX_CLK, HIGH); delayMicroseconds(1); digitalWrite(HX_CLK, LOW);
  if (count & 0x800000) count |= 0xFF000000;
  return (long)count;
```

```
}


// HARDWARE EMERGENCY ALARM (Kills relay, locks system for 20s)
void triggerAlarm(String reason) {
  if (alarmActive) return;
  digitalWrite(SYSTEM_RELAY, LOW);
  systemActive = false;
  alarmActive = true;
  alarmMessage = reason;
  alarmStartTime = millis();
  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, HIGH);
  digitalWrite(BUZZER, HIGH);
  Serial.println("\n!!! ALERT: " + reason + " !!!");
}
void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  pinMode(SYSTEM_RELAY, OUTPUT);
  pinMode(FLOW_SENSOR, INPUT_PULLUP);
```

```
pinMode(HX_DATA, INPUT_PULLUP);
pinMode(HX_CLK, OUTPUT);
pinMode(GREEN_LED, OUTPUT);
pinMode(RED_LED, OUTPUT);
pinMode(BUZZER, OUTPUT);

display.begin(SSD1306_SWITCHCAPVCC, 0x3C);

// 1. Splash Screen
display.clearDisplay();
display.setTextSize(1);
display.setTextColor(WHITE);
display.setCursor(15, 10);
display.println("ANESTHESIA BASED");
display.setCursor(10, 25);
display.println("PATIENT MONITORING");
display.setCursor(45, 40);
display.println("SYSTEM");
display.display();
delay(5000);
// 2. Finger Instruction
```

```
  display.clearDisplay();
  display.setCursor(15, 25);
  display.println("PLACE FINGER ON");
  display.setCursor(45, 40);
  display.println("SENSOR");
  display.display();
  delay(5000);
  sensors.begin();
  particleSensor.begin(Wire, I2C_SPEED_FAST);
  particleSensor.setup();
  attachInterrupt(digitalPinToInterrupt(FLOW_SENSOR), pulseCounter, FALLING);
  long sum = 0;
  for(int i=0; i<10; i++) sum += readPressureRaw();
  pressureOffset = sum / 10;
}

void loop() {
  // 1. HARDWARE RESTART LOGIC (20 Seconds after alarm)
  if (alarmActive && (millis() - alarmStartTime > 20000)) {
    alarmActive = false;
    digitalWrite(RED_LED, LOW);
```

```
  digitalWrite(BUZZER, LOW);
  Serial.println("System Restarted after 20s Auto-Timeout.");
 }
 // 2. BUTTON TOGGLE (Anesthesia Mode)
 bool currentButton = digitalRead(BUTTON_PIN);
 if (lastButtonState == HIGH && currentButton == LOW) {
  delay(100);
  if (!alarmActive) {
     systemActive = !systemActive;
     digitalWrite(SYSTEM_RELAY, systemActive ? HIGH : LOW);
     if(systemActive) { activeStartTime = millis(); }
  }
  while(digitalRead(BUTTON_PIN) == LOW);
 }
 lastButtonState = currentButton;

 // 3. HARDWARE SAFETY (Gas & Pressure)
 int gasValue = analogRead(GAS_SENSOR);
 long rawP = readPressureRaw();
 long currentP = (rawP == -999) ? 0 : (rawP - pressureOffset);
```

```
if (gasValue > GAS_THRESHOLD) {
  triggerAlarm("GAS LEAKAGE ALERT");
}

if (systemActive && (millis() - activeStartTime > 3000)) {
  if (currentP > MAX_PRESSURE) {
    triggerAlarm("HIGH PRESSURE");
  }
}

// 4. VITALS ACQUISITION
for (byte i = 0 ; i < BUFFER_SIZE ; i++) {
  if (alarmActive) break;

  while (!particleSensor.available()) particleSensor.check();
  redBuffer[i] = particleSensor.getRed();
  irBuffer[i] = particleSensor.getIR();
  particleSensor.nextSample();

  // Mid-read pressure check
  if (systemActive && (i % 25 == 0)) {
```

```
      if ((readPressureRaw() - pressureOffset) > MAX_PRESSURE) triggerAlarm("HIGH
PRESSURE");
    }
  }
  if (!alarmActive) {
    maxim_heart_rate_and_oxygen_saturation(irBuffer, BUFFER_SIZE, redBuffer, &spo2,
&validSPO2, &heartRate, &validHeartRate);
    sensors.requestTemperatures();
    float temp = sensors.getTempCByIndex(0);
    // SOFT VITALS ALERT LOGIC (Updates OLED & LEDs, DOES NOT shut down system)
    vitalStatus = "STABLE";
    if (validHeartRate) {
      if (heartRate < 60) {
        vitalStatus = "LOW VITALS";
      } else if (heartRate > 150) {
        vitalStatus = "HIGH VITALS";
      }
    } else {
      vitalStatus = "CALCULATING...";
    }
    // LED Logic for Vitals
    if (vitalStatus == "LOW VITALS" || vitalStatus == "HIGH VITALS") {
```

```
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(RED_LED, HIGH);
  } else {
    digitalWrite(GREEN_LED, HIGH);
    digitalWrite(RED_LED, LOW);
  }
}

// 5. OLED DISPLAY UPDATE
display.clearDisplay();
display.setTextSize(1);

if (alarmActive) {
  // Hard Alarm Screen (Pressure/Gas)
  display.setCursor(10, 20);
  display.println("!!! ALERT !!!");
  display.setCursor(10, 35);
  display.println(alarmMessage);
} else {
  // Normal Running Screen
  display.setCursor(0,0);
```

```
  display.print(systemActive ? "ANESTHESIA: ON" : "ANESTHESIA: OFF");
  display.setCursor(0,12);
  display.print("P: "); display.print(currentP); display.print(" G: "); display.print(gasValue);
  display.setCursor(0,24);
  display.print("BPM: "); display.print(validHeartRate ? String(heartRate) : "--");
  display.print(" SpO2: "); display.print(validSPO2 ? String(spo2) : "--%");
  display.setCursor(0,36);
  display.print("Temp: "); display.print(sensors.getTempCByIndex(0), 1); display.print(" C");
  display.setCursor(0,48);
  display.print("Status: "); display.print(vitalStatus);
}
display.display();

// 6. SERIAL MONITOR
noInterrupts();
long pulses = pulseCount; pulseCount = 0;
interrupts();
float flow = pulses / FLOW_CAL_FACTOR;
if (!alarmActive) {
  Serial.print("P: "); Serial.print(currentP);
  Serial.print(" | F: "); Serial.print(flow, 2); // Flow kept only in Serial
```

```
Serial.print(" | HR: "); Serial.print(validHeartRate ? String(heartRate) : "--");

Serial.print(" | SpO2: "); Serial.println(validSPO2 ? String(spo2) : "--");
```

REFERENCES

[1] J. G. Webster, *Medical Instrumentation: Application and Design*, 4th Edition, Wiley, 2010.

[2] John Allen, "Photoplethysmography and its application in clinical physiological measurement," *Physiological Measurement*, vol. 28, no. 3, pp. R1–R39, 2007.

[3] World Health Organization (WHO), *Pulse Oximetry Training Manual*, WHO Press, 2011.

[4] Espressif Systems, *ESP32 Technical Reference Manual*, 2023.

[5] Maxim Integrated, *MAX30102 Pulse Oximeter and Heart Rate Sensor Datasheet*, 2022.

[6] Texas Instruments, *LM2596 SIMPLE SWITCHER Power Converter Datasheet*, 2021.

[7] Arduino, "Arduino Documentation," Available: https://www.arduino.cc

[8] D. Zhang, H. Wang, and Y. Li, "IoT-Based Smart Healthcare Monitoring System," *IEEE Access*, vol. 8, pp. 123–130, 2020.

[9] Hanwei Electronics, *MQ-135 Gas Sensor Datasheet*, 2020.

[10] A. Pantelopoulos and N. G. Bourbakis, "A Survey on Wearable Sensor-Based Systems for Health Monitoring and Prognosis," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 40, no. 1, pp. 1–12, 2010.

[11] B. G. L. Healey and R. W. Picard, "Detecting stress during real-world driving tasks using physiological sensors," IEEE Trans. Intell. Transp. Syst., vol. 6, no. 2, pp. 156–166, Jun. 2005

[12] S. Rajasekaran, V. Subramanian, and P. Venkatesan, "Wireless health monitoring system using IoT," Int. J. Eng. Technol., vol. 7, no. 2, pp. 123–128, 2018.

[13] NXP Semiconductors, "PCF8574 Remote 8-bit I/O Expander for I²C Bus," Datasheet, Rev. 7, 2015.

[14] D. A. Patterson and J. L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, 5th ed. San Francisco, CA: Morgan Kaufmann, 2014.

[15] Microchip Technology Inc., "ATmega328P 8-bit AVR Microcontroller Datasheet," Rev. 7810D, 2018.